Proiectarea sistemelor secvenţiale

Principii

- modelarea - în principal comportamentală
  - se pot utiliza şi elemente structurale pentru unele componente
- trebuie luat în considerare semnalul de ceas
  - în mod explicit

Exemplu 1

- bistabil D - latch

```
module Latch(Clk,D,Q);
input Clk,D;
output Q;
reg Q;
always @(Clk,D)
  if(Clk==1) Q<=D;
endmodule
```

Exemplu 2

- bistabil D - flip-flop
- considerăm şi intrările R şi S
  - asincrone (nu depind de semnalul de ceas)
  - prioritare
  - necesare pentru iniţializare şi alte operaţii
- tratarea semnalului de ceas
  - contează doar frontul crescător
  - trebuie detectat

Exemplu 2 - varianta 1

```
module FlipFlop(Clk,D,R,S,Q);
input Clk,D,R,S;
output Q;
reg Q;
always @(posedge Clk,D,R,S)
  if(R==1) Q<=0;
  else if(S==1)
     Q<=1;
```

Exemplu 2 - varianta 1 (cont.)

```
  else if(posedge Clk)
     Q<=D;
endmodule
```

Exemplu 2 - varianta 1 (cont.)

- detectarea frontului crescător - `posedge`
  - front descrescător - `negedge`
- varianta NU ESTE CORECTĂ
  - frontul crescător poate fi precizat în lista de sensibilitate a structurii `always`
  - dar nu poate fi testat într-o instrucţiune `if`
  - pot fi testate doar valori ale semnalelor, nu şi tranziţii ale acestora

Exemplu 2 - varianta 2

```
module FlipFlop(Clk,D,R,S,Q);
input Clk,D,R,S;
output Q;
reg Q;
always @(R,S)
  if(R==1) Q<=0;
  else if(S==1)
     Q<=1;
```

Exemplu 2 - varianta 2 (cont.)

```
always @(posedge Clk)
  if(R==0 && S==0)
     Q<=D;
endmodule
```

Exemplu 2 - varianta 2 (cont.)

- se pot folosi 2 sau mai multe structuri `always`
  - şi `initial`
- toate se execută în paralel
- intrările R şi S trebuie să fie prioritare
  - sunt testate în cealaltă structură `always`
- combinaţiile sincron-asincron - dificil de tratat

Automate

- două componente
  - evoluţia stării
    - pe baza semnalului de ceas
  - elemente de acţionare
    - descriere de obicei comportamentală
    - calculează valorile pentru starea următoare şi ieşirile
- izolare - structuri `always` separate
  - modularitate - mai uşor de înţeles şi controlat

Structura de bază

```
module nume_modul(Clk,intrari,iesiri);
input Clk,intrari;
output iesiri;
reg iesiri;
reg stare_curenta,stare_urmatoare;
always @(posedge Clk)
  stare_curenta<=stare_urmatoare;
  // eventual reset, ...
```

Structura de bază (cont.)

```
always @(intrari,stare_curenta)
  begin
  // calcul iesiri, stare urmatoare
  end
endmodule
```

Structura de bază (cont.)

- partea combinaţională răspunde imediat la orice modificare a intrărilor sale
  - starea curentă
  - intrări
- partea secvenţială permite modificarea doar pe frontul crescător al ceasului
- nu e obligatoriu ca `stare_curenta` şi `stare_urmatoare` să fie implementate ca regiştri separaţi

Exemplu

- măsurarea perioadei unui semnal
- digitizare - transformare într-un semnal cu valori doar 0 şi 1
  - perioada rămâne aceeaşi
  - nu este sigur că valorile de 0 şi respectiv 1 au aceeaşi durată în cadrul unei perioade
- aceleaşi obiective ca la cursul 4

Varianta 1

```
module P(start,i,digit,print);
input start,i;
output reg [3:0]digit;
output reg print;
always @(start,i)
begin
  digit=0;
  print=0;
```

Varianta 1 (cont.)

```
if(i==0) begin
   while(i==0) ;
   while(i==1) digit=digit+1;
   while(i==0) digit=digit+1;
   print=1;
end
else begin
   while(i==1) ;
```

Varianta 1 (cont.)

```
    while(i==0) digit=digit+1;
    while(i==1) digit=digit+1;
    print=1;
  end
end
endmodule
```

Varianta 1 (cont.)

- problema
  - nu apare semnalul de ceas
  - la ce momente au loc toate acţiunile?
- semnalul de ceas trebuie să apară explicit ca intrare
- o instrucţiune poate fi întârziată până la apariţia unui eveniment
  - ex.: `@(posedge clk) digit=digit+1;`

Varianta 1 (cont.)

- o soluţie ar fi deci ca toate acţiunile să fie întârziate în acest mod

```
while(i==0) @(posedge clk);
while(i==1)
   @(posedge clk) digit=digit+1;
while(i==0)
   @(posedge clk) digit=digit+1;
```

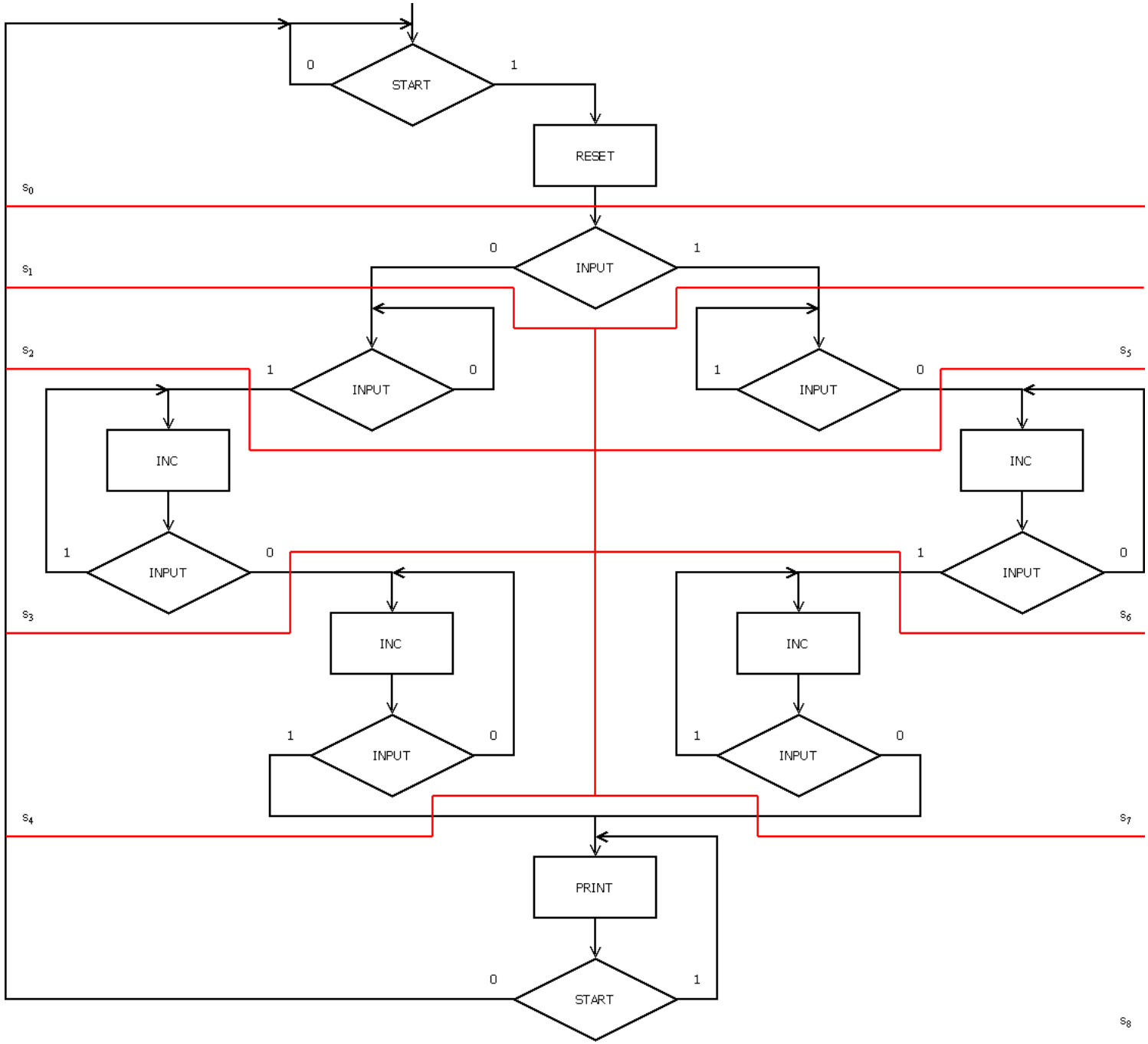
- foarte greu de gestionat

Varianta 2

- revenim la proiectarea automatului
  - la fel ca la cursul 4
  - orientativ - ne interesează stările, nu exact semnalele de comandă
    - implementarea acestora o realizează compilatorul
    - ex.: în loc de RESET vom scrie

```
digit<=0; display<=0;
```

    - descrierea - comportamentală


START
0
1
RESET
s_0
INPUT
0
1
s_1
INPUT
1
0
s_2
INPUT
1
0
s_5
INC
INC
INPUT
1
0
INPUT
1
0
s_3
s_6
INC
INC
INPUT
1
0
INPUT
1
0
s_4
s_7
PRINT
START
0
1
s_8

Varianta 2 - implementare

```
module P(Clk,start,i,digit,print);
input Clk,start,i;
output reg [3:0]digit;
output reg print;
reg [3:0]state;
reg [3:0]state_next;
always @(posedge Clk)
  state<=state_next;
```

Varianta 2 - implementare (cont.)

```
always @(start,i,state)
  if(state==0)
  begin
     if(start==0) state_next<=0;
     else state_next<=1;
     digit<=0;
     print<=0;
  end
```

Varianta 2 - implementare (cont.)

```
  else if(state==1) begin
     if(i==1) state_next<=2;
     else state_next<=5;
  end
  ... // celelalte stari
initial
  state<=0; state_next<=0;
endmodule
```

Modelarea prin automate (1)

- limbajul Verilog este util mai ales pentru partea de implementare
- descrierea algoritmului de funcţionare rămâne sarcina proiectantului
  - modelare pe bază de stări (automat)
- atenţie la atribuirea de valori pentru ieşiri
  - teoretic - toate ieşirile primesc valori la fiecare stare
  - practic - atunci când se schimbă valorile

Modelarea prin automate (2)

Avantaj

- separă partea de memorare a stării (secvenţială) de cea combinaţională
  - două blocuri `always` independente
    - unul actualizează starea
    - celălalt realizează calculul valorilor pentru ieşiri şi starea următoare
  - descriere clară

Modelarea prin automate (3)

Probleme

- la descrierea părţii combinaţionale
  - blocul `always` reacţionează la modificarea unei variabile
  - nu este potrivit pentru integrarea într-un sistem secvenţial

Exemplu

```
module ex(Clk,x,y,z);
input Clk,x,y;
output z;
reg z;
//starea - cati biti sunt necesari
reg [2:0]state,state_next;
always @(posedge Clk)
  state<=state_next;
```

Exemplu (cont.)

```
always @(x,y,state)
  if(state==0)
     if(x==1) begin
          z<=z+1;
          state_next<=0;
     end
     else state_next<=1;
  ...
```

Problema 1

- starea 0 - se incrementează `z` cât timp `x` are valoarea 1
  - buclă de program
- suntem în starea 0 şi starea următoare este tot 0
- dacă `x` şi `y` nu variază, `z` nu e incrementat
  - `always` nu detectează vreo schimbare pe nici una din variabile, deci nu se activează

Problema 2

- starea 0 - apare o modificare a valorii variabilei `y`
- nu ar trebui să influenţeze cu nimic evoluţia sistemului
  - deoarece `y` nu intervine în starea 0
  - dar `always` detectează o schimbare pe una din variabile, deci se activează
  - ca urmare, `z` are o incrementare parazită

Soluţii (1)

- `always` să reacţioneze şi la schimbările semnalului de ceas
  - sau numai la frontul crescător al acestuia
  - pot apărea interferenţe cu blocul `always` care se ocupă de actualizarea stării
    - atribuirile neblocante (<=) se execută în paralel şi atunci când sunt în blocuri `always/initial` diferite
  - nu rezolvă a doua problemă

Soluţii (2)

- toate variabilele care apar în lista de sensibilitate a blocului `always` să fie testate prin instrucţiuni `if` în toate stările
  - s-ar rezolva a doua problemă
  - foarte greu de realizat practic

Soluţii (3)

- descriere structurală (de nivel jos) a părţii combinaţionale
  - fără blocuri `always`
  - ecuaţii booleene şi/sau circuite predefinite
  - ar rezolva ambele probleme
  - puterea limbajului Verilog se pierde în foarte mare măsură dacă renunţăm la descrierile comportamentale

Soluţii (4)

- un singur vector de stare (în loc de doi)
- un singur bloc `always`
  - activat doar de frontul crescător al semnalului de ceas
  - toate calculele se (re)fac pe frontul crescător
  - deci toate variabilele sunt utilizate şi actualizate numai pe frontul crescător
- ambele probleme sunt rezolvate

Implementare

```
module nume_modul(Clk,intrari,iesiri);
input Clk,intrari;
output iesiri;
reg iesiri;
reg stare;
always @(posedge Clk)
  if(stare==0)
     //descriere stare 0
```

Implementare (cont.)

```
    //actualizare stare
  else if(stare==1)
    //descriere stare 1
    //actualizare stare
  ... //alte stari
endmodule
```

Limitare

- pentru variabilele de intrare se iau în considerare doar valorile de la momentele când apare un front crescător al ceasului
  - dacă o variabilă de intrare este activată doar între două fronturi crescătoare consecutive, va fi ignorată
  - în general nu este o problemă
    - variabilele de intrare variază lent
    - la nevoie, intrările care provin de la alte circuite pot fi sincronizate după acelaşi semnal de ceas

Exemplu

- calculul celui mai mare divizor comun a două numere
- algoritmul lui Euclid
- numerele sunt primite simultan din exterior
- semnale de notificare
  - `start` - numerele sunt disponibile la intrare
  - `ack` - rezultatul este disponibil la ieşire

Implementare

```
module
  cmmdc(Clk,start,x,y,r,ack);
input Clk,start;
input [7:0]x,y;
output reg [7:0]r;
output reg ack;
reg [7:0]a,b;
reg [1:0]state;
```

Implementare (cont.)

```
always @(posedge Clk)
  if(state==0)
     if(start==1) begin
          a<=x;
          b<=y;
          ack<=0;
          state<=1;
     end
     else state<=0;
```

Implementare (cont.)

```
else if(state==1)
   if(a==b) begin
        r<=a;
        state<=2;
   end
   else if(a>b) begin
        a<=a-b;
        state<=1;
   end
```

Implementare (cont.)

```
    else begin
        b<=b-a;
        state<=1;
    end
else if(state==2) begin
        ack<=1;
        state<=0;
    end
```

Implementare (cont.)

```
initial
begin
  state<=0;
  ack<=0;
end
endmodule
```